



PEC4: Introducción al aprendizaje computacional

Presentación

Cuarta PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Conocer los diferentes aspectos del aprendizaje computacional (aprendizaje supervisado, no supervisado y por refuerzo).
- Conocer los fundamentos teóricos de los métodos más representativos.
- Conocer el tratamiento de los conjuntos de datos para la correcta validación de los sistemas de aprendizaje.

Objetivos

Esta PEC pretende evaluar diferentes aspectos de aprendizaje supervisado y no supervisado.

Descripción de la PEC a realizar

Pregunta 1 (4 puntos)

Se quiere automatizar el sistema de permisos de vuelo en un pequeño aeródromo. A partir de un conjunto de observaciones de las condiciones meteorológicas relativas a la temperatura, humedad y viento momentos antes del intento de despegue de 14 vuelos se quiere construir un sistema que pueda ayudar a tomar la decisión de si un vuelo debe despegar.



En este ejercicio, debéis construir un árbol de decisión con las tres características que tenemos en la tabla siguiente: **Temperatura**, que puede tomar los valores: Hot (caliente), Mild (moderado), Cool (fresco). **Humedad**, que puede tomar los valores: High (alto), Normal (moderado) y **Viento** que puede tomar los valores: Strong (fuerte), Weak (débil).

La columna vuelo es el objeto de clasificación.

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
1	Hot	High	Weak	No
2	Hot	High	Strong	No
3	Hot	High	Weak	Yes
4	Mild	High	Weak	Yes
5	Cool	Normal	Weak	Yes
6	Cool	Normal	Strong	No
7	Cool	Normal	Strong	Yes
8	Mild	High	Weak	No
9	Cool	Normal	Weak	Yes
10	Mild	Normal	Weak	Yes
11	Mild	Normal	Strong	Yes
12	Mild	High	Strong	Yes
13	Hot	Normal	Weak	Yes
14	Mild	High	Strong	No



¿Qué recomendación realizarías a los siguientes vuelos? ¿Deberían despegar?

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO
1s	Hot	Normal	Strong
2s	Hot	High	Weak
3s	Mild	High	Strong

Solución

Para construir el árbol seguimos un proceso iterativo, en el que calculamos la bondad de los atributos. Es decir, lo bien clasificada que nos queda la clase vuelo según la característica que elegimos. En la tabla vemos el cálculo:

Temperatura	Yes	No	Bondad
Hot	2	2	
Cool	4	2	
Mid	3	1	$(2+4+3) / 14 = 0,642$
Humedad			
High	3	4	
Normal	6	1	$(6+4) / 14 = 0,714$
Viento			
Weak	6	2	
Strong	3	3	$(6+3) / 14 = 0,642$

Podemos ver que la **humedad** es la característica que mejor nos separa la clase objetivo (vuelo). A continuación, deberemos evaluar las dos ramas que obtenemos de esta partición. Construiremos el árbol en profundidad y luego procederemos a calcular la rama izquierda.

Si elegimos la humedad = High como rama izquierda:

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
1	Hot	High	Weak	No
2	Hot	High	Strong	No
3	Hot	High	Weak	Yes
4	Mild	High	Weak	Yes
8	Mild	High	Weak	No
12	Mild	High	Strong	Yes
14	Mild	High	Strong	No



Entonces el cálculo de las bondades es el siguiente:

Temperatura	Yes	No	Bondad
Hot	1	2	
Cool	-	-	
Mid	2	2	$(2+2) / 7 = 0,571$
Viento			
Weak	2	2	
Strong	1	2	$(2+2) / 7 = 0,571$

En caso de empate podríamos elegir cualquiera de los dos, nosotros elegiremos la característica **viento**. En primer lugar, veremos el subconjunto de viento Weak.

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
1	Hot	High	Weak	No
3	Hot	High	Weak	Yes
4	Mild	High	Weak	Yes
8	Mild	High	Weak	No

En este caso nos quedan dos nodos hoja con una probabilidad del 50 % cada uno. No podemos seguir realizando más particiones ya que hemos utilizado todas las características disponibles.

Ahora analizaremos la rama que se genera con el subconjunto que tienen viento Strong:

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
2	Hot	High	Strong	No
12	Mild	High	Strong	Yes
14	Mild	High	Strong	No

En este caso nos quedan dos nodos hoja, si tenemos temperatura Mild tenemos una probabilidad del 50% en cambio si tenemos temperatura Hot no deberíamos volar.

Ahora, vamos a calcular la rama derecha, es decir si humedad = Normal:

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
5	Cool	Normal	Weak	Yes
6	Cool	Normal	Strong	No
7	Cool	Normal	Strong	Yes
9	Cool	Normal	Weak	Yes



10	Mild	Normal	Weak	Yes
11	Mild	Normal	Strong	Yes
13	Hot	Normal	Weak	Yes

Entonces el cálculo de las bondades es el siguiente:

Temperatura	Yes	No	Bondad
Hot	1	0	
Cool	3	1	
Mid	2	0	$(1+3+2) / 7 = 0,857$
Viento			
Weak	4	0	
Strong	2	1	$(4+2) / 7 = 0,857$

En caso de empate podríamos elegir cualquiera de los dos, nosotros elegiremos la característica **temperatura**.

En este caso nos quedarán tres subárboles. En el primer subárbol, el que se crea con la característica de temperatura = Cool tenemos las siguientes observaciones:

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
5	Cool	Normal	Weak	Yes
6	Cool	Normal	Strong	No
7	Cool	Normal	Strong	Yes
9	Cool	Normal	Weak	Yes

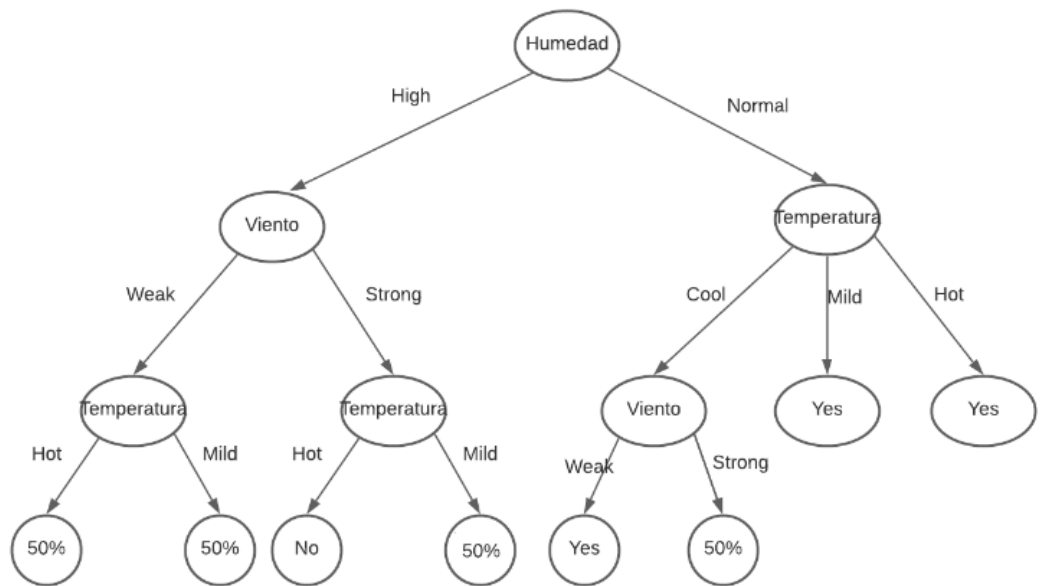
Esto resulta en dos nodos hoja, con viento Strong (50%) y con viento Weak (Yes). El siguiente subárbol (temperatura = Mild) es un nodo hoja con valor Yes.

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
10	Mild	Normal	Weak	Yes
11	Mild	Normal	Strong	Yes

En este caso sucede exactamente como el anterior, tenemos un nodo hoja

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO	VUELO
13	Hot	Normal	Weak	Yes

En el siguiente esquema podemos ver el árbol de decisión construido.



¿Qué recomendación realizarías a los siguientes vuelos? ¿Deberían despegar?

ÍNDICE	TEMPERATURA	HUMEDAD	VIENTO
1s	Hot	Normal	Strong
2s	Hot	High	Weak
3s	Mild	High	Strong

Para responder esta pregunta es suficiente con interpretar el árbol de decisión de manera correcta y seguir las decisiones que este nos indica:

1S - Es un vuelo que puede despegar con total seguridad.

2S - Es un vuelo en el que no podemos tomar la decisión basándonos en el árbol, necesitaríamos otros criterios adicionales.

3S - Es un vuelo que puede despegar con total seguridad.



Ejercicio 2 (4 puntos)

Tal como ya sabemos, el uso de una red neuronal puede ser más adecuado cuando los atributos de nuestro problema son numéricos y no categóricos como en el ejercicio anterior. Hemos diseñado y entrenado una red neuronal a partir de las observaciones de dos sensores que controlan la apertura de una puerta. Los sensores pueden tomar valores enteros entre -10 y 10. La salida de la red puede ser 0 (cerrar puerta) o 1 (abrir puerta).

Queremos comprobar la validez de nuestro clasificador con el siguiente conjunto de datos:

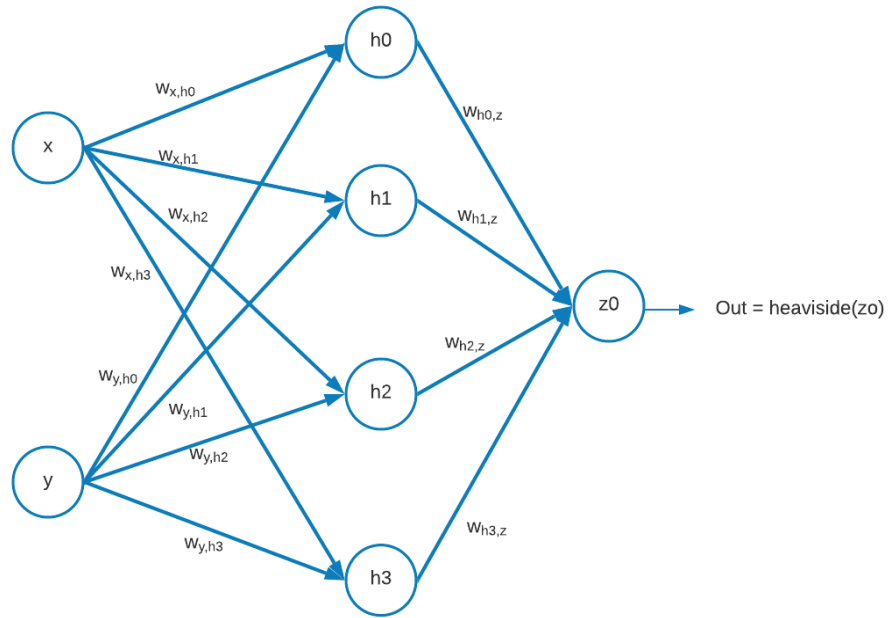
Sensor1	Sensor2	Class
2	-1	1
2	-2	1
2	2	1
2	1	1
2	-3	1
-1	-5	0
-1	4	0
-1	-2	0
-1	-2	0
4	-4	1
7	4	1
-3	3	0
-1	9	0
-3	1	0
1	0	1

Dada la siguiente red neuronal ya entrenada donde las neuronas x e y forman la capa de entrada en la que introduciremos los datos del Sensor1 y el Sensor2 respectivamente.

Calcula la salida de la red para cada observación, la matriz de confusión de este clasificador y la métrica F1. Para calcular la matriz de confusión y la



métrica F1 consideraremos 1 como el valor positivo y 0 como el valor negativo.



Donde:

$h0 = w_{x,h0}x + w_{y,h0}y$
$h1 = w_{x,h1}x + w_{y,h1}y$
$h2 = w_{x,h2}x + w_{y,h2}y$
$h3 = w_{x,h3}x + w_{y,h3}y$
$z0 = w_{h0,z}h0 + w_{h1,z}h1 + w_{h2,z}h2 + w_{h3,z}h3$

Los pesos asociados a cada una de las conexiones neuronales está definido en la siguiente tabla:

W	h0	h1	h2	h3
x	0	-1	0	1
y	-1	0	-1	-1
z	0	-1	0	1

La siguiente función *heaviside* que depende del valor de entrada:

$$heaviside(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ 1, & \text{if } x > 0 \end{cases}$$



Solución

En la siguiente tabla tenemos los cálculos intermedios necesarios, y la salida de la red, Out:

x	y	h0	h1	h2	h3	z	Out	Class
2	-1	1	-2	1	3	5	1	1
2	-2	2	-2	2	4	6	1	1
2	2	-2	-2	-2	0	2	1	1
2	1	-1	-2	-1	1	3	1	1
2	-3	3	-2	3	5	7	1	1
-1	-5	5	1	5	4	3	1	0
-1	4	-4	1	-4	-5	-6	0	0
-1	-2	2	1	2	1	0	0	0
-1	-2	2	1	2	1	0	0	0
4	-4	4	-4	4	8	12	1	1
7	4	-4	-7	-4	3	10	1	1
-3	3	-3	3	-3	-6	-9	0	0
-1	9	-9	1	-9	-10	-11	0	0
-3	1	-1	3	-1	-4	-7	0	0
1	0	0	-1	0	1	2	1	1

La matriz de confusión es la siguiente:

Predicción \ Verdad	Positivo (1)	Negativo (0)
Positivo (1)	8 (TP)	1 (FP)
Negativo (0)	0 (FN)	6 (TN)

A continuación, el cálculo de las medidas:

$$\text{Precisión} = 0,89$$

$$\text{Sensibilidad} = 1$$

$$F1 = (2 * \text{Precisión} * \text{Sensibilidad}) / (\text{Precisión} + \text{Sensibilidad}) = 0,94$$



Pregunta 3 (2 puntos)

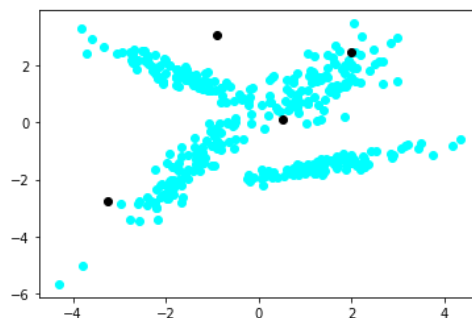
El notebook de Python adjunto genera un conjunto de 400 puntos en un espacio 2D. Además, podréis encontrar implementado prácticamente en su totalidad el algoritmo de K-means suponiendo una $K=4$.

Se pide que:

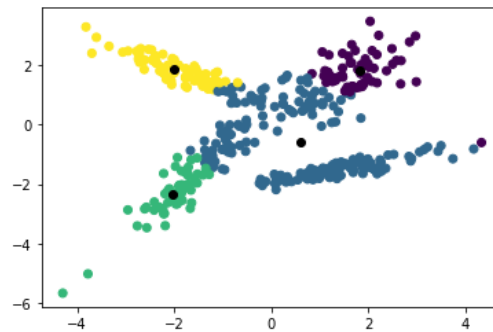
- **3.a.1 Implementéis la función *minimum_distance*** que recibe por parámetro una lista con los centroides y un punto. Esta función devuelve el índice del centroide al cual el punto tiene una distancia euclídea menor.
- **3.a.2 Añadáis un criterio de finalización.** El criterio de finalización del algoritmo que está implementado es el número de iteraciones (variable *max_iterations*) debéis añadir un criterio de parada si ninguno de los centroides se mueve más de 0.01 unidades respecto a los centroides de la iteración anterior. ¿Cuántas iteraciones nos ahorramos con este nuevo criterio?
- **3.a.3 Cread una lista llamada *y*, que contiene el índice del centroide más próximo a cada muestra del conjunto de datos.** Ejecutando la celda siguiente podréis ver los resultados de forma gráfica.
- **3.a.4 Volved a ejecutar el k-means para $K = 2, 3$ y 5 .**

Como resultado de este ejercicio, explicar el código realizado y copiad capturas de pantalla de los resultados obtenidos mediante el *clustering* para cada K .

A modo de ejemplo, para este conjunto de datos (diferente al que tendréis vosotros):



Se pide un resultado similar al siguiente, en el que se pueden observar los diferentes clusters y los centroides:



Nota: En el script encontraréis marcadas las zonas del código que se corresponden con los apartados 3.a.1, 3.a.2. y 3.a.3

Solución

3.a.1

Tenéis un ejemplo de implementación de la función ***minimum_distance*** en el script de la solución.

3.a.2

El código usado para implementar el nuevo criterio se encuentra en el script de la solución. El número de iteraciones ha pasado de 2000 a 6.

Recursos

Para hacer esta PEC el material imprescindible es el módulo 5. Para ejecutar los notebooks en Python del ejercicio se recomienda usar Google Colab que es gratuito (<https://colab.research.google.com>) y provee de todo lo necesario para su realización.

Criterios de valoración

Las puntuaciones se muestran en cada pregunta del enunciado.

Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo PDF usando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser *Apellidos_Nombre_IA_PEC4* con la extensión .pdf (PDF).



La fecha límite de entrega es el: **23/05/2021** (a las 24 horas).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...).

El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente..